

Git 与 GitHub

从第一次提交，
到一次完整协作。

本地版本控制 / GitHub 托管 / 分支与 PR / 基础安全



学习路线：从本地记录到远程协作

以纯文本项目 `git-class-demo` 串联完整流程。

01

留下版本

初始化 → 暂存 → 提交
比较两次修改
在本地分支里工作

02

完成协作

新建 GitHub 仓库
push / clone / pull
发起并合并一个 PR

03

守住边界

忽略本地凭据
理解历史不会自动消失
检查账号与仓库权限

本地管理版本，远程共享提交，通过分支和 Pull Request 协作。

版本控制解决什么问题？

版本控制让“谁改了什么、为什么改”成为可以追溯的记录。

只靠文件名

文件命名示例

```
report_final.txt  
report_final_v2.txt  
report_really_final.txt
```

很难看出差异，也容易覆盖同伴的修改。

把变化留在历史里

- C1 建立项目
- C2 更新学习笔记
- C3 加入介绍文件

不是多复制几份文件，而是让每个有意义的改动留下可比较的记录。

Git $\neq$ GitHub

一个负责版本记录，一个提供远程托管与协作。

在你的电脑上

Git

保存提交与分支
大部分本地操作无需联网

在远程服务上

GitHub

托管 Git 仓库
提供 PR、评审等协作界面

commit 是本地提交；**push** 才把对应分支的提交送往远程。

先记住这四个位置

每条命令都有“起点”和“终点”。



查看变化： `git status` → `git diff` → `git diff --cached` 。先看清，再提交。

环境准备与命令约定

Windows 使用 Git Bash；macOS 与 Linux 使用系统终端。

三条检查命令

```
git --version  
ssh -V  
pwd
```

1

Windows：Git Bash

示例写文件语法不直接用于 CMD / PowerShell。

2

macOS / Linux：Terminal

示例命令使用 zsh / bash 语法。

参数： YOUR_USERNAME 为仓库账号； YOUR_NOREPLY_EMAIL 为提交邮箱。

环境要求： Git 2.28+。使用新建练习目录，GitHub 操作需要账号认证与仓库权限。

01 | 初始化本地仓库

位置：用户主目录下的全新 git-class-demo。

创建仓库与配置作者

```
cd ~  
mkdir git-class-demo  
cd git-class-demo  
git init -b main  
git config user.name "Git Demo"  
git config user.email "YOUR_NOREPLY_EMAIL"
```

1 init

初始化当前目录，主分支叫 main。

2 config

作者信息，不是 GitHub 登录。

同名目录已存在时应换名；不加 `--global` 的配置只作用于当前仓库。

02 | 建立示例项目

只用文本文件；所有凭据字段都是没有真实权限的占位值。

项目目录结构

```
git-class-demo/  
  README.md      project description  
  notes.txt       learning notes  
  .gitignore      ignore rules  
  .env.example    placeholders only  
  .env            local fake value
```

这次提交四个文件

README、notes、
.gitignore、.env.example

.env 不提交

`git status --short` 中的 `??` 表示未跟踪文件；忽略规则命中的 `.env` 不显示。

03 | 暂存与第一次提交

明确选择文件，并检查将要写入历史的内容。

暂存、检查与提交

```
git add README.md notes.txt .gitignore .env.example
git status --short
git diff --cached
git commit -m "Initial commit"
git status
git --no-pager log --oneline
```

commit 只写入本地仓库；工作区干净并不代表已经同步到 GitHub。

04 | 修改文件与查看差异

工作区的变化，与下一次提交的内容，分开观察。

修改与提交

```
printf 'I can track changes.\n' >> notes.txt
git diff
git add notes.txt
git diff --cached
git commit -m "Update learning notes"
git --no-pager log --oneline
```

先 diff

工作区 ↔ 暂存区

再 --cached

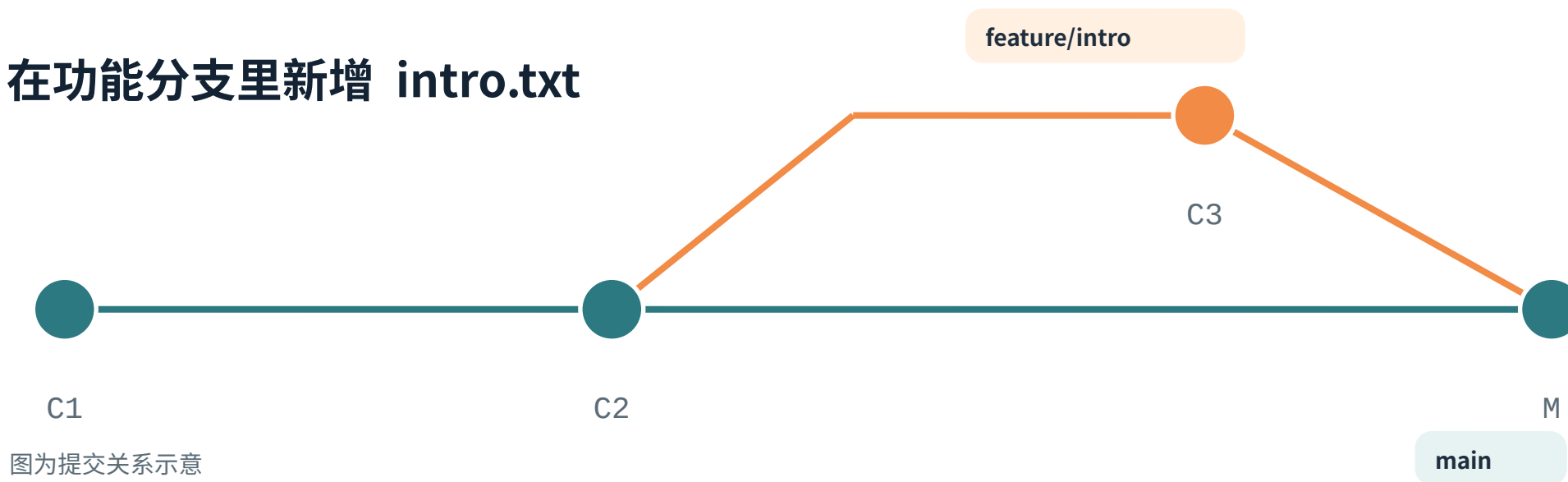
暂存区 ↔ 当前提交

add 之后再编辑，不会自动更新暂存快照。要提交新改动，就再次 add。

分支：先独立做，再决定如何合入

分支是指向提交的轻量引用，不是复制一整个项目目录。

在功能分支里新增 intro.txt



图为提交关系示意

合并的目标是“当前分支”。先 switch main，再 merge 功能分支。

05 | 创建分支并合并

观察：切回 main 时，刚才新增的文件不在当前版本里。

创建、切换与合并

```
git switch -c feature/intro
printf 'Hello from a feature branch.\n' > intro.txt
git add intro.txt
git commit -m "Add introduction"
git switch main
ls
git merge --no-ff feature/intro -m "Merge intro"
git --no-pager log --oneline --graph --all
```

--no-ff 保留合并提交； **-m** 指定这次合并的提交说明。

上 GitHub 前，分清三种“身份”

配置作者信息，并不会给你任何仓库的写权限。

01

提交作者

user.name / user.email
写入提交记录

02

认证方式

SSH 密钥或 HTTPS 凭据
证明“你是谁”

03

仓库权限

owner / repo / 写权限
决定“你能改哪里”

```
ssh -T git@github.com
```

已认证 + 用户名正确 = 身份检查通过

私钥只留本机，GitHub 只添加 .pub；首次连接核对官方主机指纹。

06 | 创建空的 GitHub 仓库

路线：GitHub → New repository。

Create a new repository • 界面示意

Owner	YOUR_USERNAME
Repository name	git-class-demo
Visibility	Private
README / .gitignore / License	不初始化 / None

1 个人账号

Owner 应与目标仓库所属账号一致。

2 保持空仓库

本地已经有首次提交；不要再生成另一套起点。

另一条路线可以“网页先建 README → clone”；不要混用两次独立初始化。

07 | 配置远程并首次推送

把 YOUR_USERNAME 替换为实际仓库 owner；git@ 不要修改。

关联远程与推送

```
git remote add origin git@github.com:YOUR_USERNAME/git-class-demo.git
git remote -v
git push -u origin main
git branch -vv
```

origin

远程地址的常用别名

main

这次推送的本地分支

-u

建立上游跟踪关系

确认推送结果，并克隆仓库

从仓库页面与本地克隆两个角度检查同步结果。

仓库页面检查

- ✓ main 分支正确
- ✓ 文件与提交说明出现
- ✓ 短哈希与本地一致
- ✓ .env 没有出现在列表

克隆到新的本地目录

```
cd ~  
git clone git@github.com:YOUR_USERNAME/\  
git-class-demo.git git-class-clone  
cd git-class-clone  
cat notes.txt  
cd ~/git-class-demo
```

clone 是建立另一份本地仓库；完成检查后，返回原来的 **git-class-demo**。

08 | 拉取远程更新

先在 GitHub 的 main 上修改 notes.txt 并提交。

GitHub 网页操作

打开 notes.txt
点击编辑
末尾加一行
Commit changes

Edited on GitHub.

本地同步命令

```
cd ~/git-class-demo  
git switch main  
git status  
git pull --ff-only  
cat notes.txt
```

pull = 先获取，再整合。--ff-only 遇到分叉会停止，不会强制覆盖本地。

09 | 推送功能分支

feature/checklist：给项目增加一份提交前检查清单。

创建并推送检查清单

```
git switch -c feature/checklist
printf '%s\n' 'Check status before commit.' \
  'Review diff before push.' > checklist.txt
git add checklist.txt
git diff --cached
git commit -m "Add demo checklist"
git push -u origin feature/checklist
```

此时 main
还没有改变

push 了功能分支
≠
已经合并主分支

PR 是合并请求；git pull 是拉取命令。名字相似，但不是同一个操作。

10 | 创建、审阅并合并 Pull Request

Pull requests → New pull request。

base: main

接收修改



compare: feature/checklist

1 Files changed

检查是否只有预期文件。

2 Create pull request

说明改了什么、为什么改。

3 确认合并

选择 Create a merge commit。

团队协作应先满足评审与仓库规则，不应绕过既有分支保护。

同步合并后的 main 分支

网页已经合并，不代表你的本地文件自动更新。

拉取合并结果

```
git switch main
git pull --ff-only
cat checklist.txt
git --no-pager log --oneline --graph --all -8
git branch -d feature/checklist
```

协作闭环

开分支
提交并推送
PR 评审合并
同步 main

-d 只安全删除本地已合并分支；若用了 **squash**，删除受阻就先保留。

安全 01 | 凭据不进库，从第一天开始

私有仓库不是保险箱，.gitignore 也不是加密。

.gitignore

```
.env  
.env.*  
!.env.example  
*.log
```

提交前检查

```
git check-ignore -v .env  
git ls-files -- .env  
git diff --cached
```

.env.example 只留字段名与占位值。
提交前检查 diff；真实 Token、私钥和密码不得写入仓库。

已跟踪文件不会因加入 .gitignore 自动消失；历史中的凭据也不会被它抹去。

安全 02 | 删掉文件，不等于删掉历史

使用独立本地仓库与无效占位值，观察文件删除前后的历史。

当前版本

.env 已经被删除

看不到文件 $\neq$ 历史没有记录

历史读取示例

```
git show HEAD~1:.env
```

```
API_KEY=DEMO_ONLY_NOT_A_REAL_KEY
```

真实泄露：先撤销 / 轮换凭据，再评估影响、协调清理历史与副本。

安全 03 | 管好账号，也管好分享范围

公开之前，再检查一遍你究竟在分享什么。

账号

保护登录

启用双因素认证
妥善保存恢复方式
私钥与恢复码不外传

仓库

最小权限

练习仓库使用 Private
只邀请必要的协作者
不把秘密写进仓库

课程

遵守要求

真实课程作业不公开
共享范围听任课老师要求
克隆代码不等于信任代码

课程作业的提交与共享范围以任课老师要求为准；练习仓库不用于公开真实课程答案。

完整流程回顾

把“文件管理”变成“有记录、有审查的协作”。

日常提交前的检查顺序

```
status → diff → add → diff --cached → commit → push
```

协作：分支 → PR → 合并 → pull

安全：凭据不进库

课后练习：建私有练习仓库，完成两次提交、一次推拉和一个 PR。不要上传真实课程答案。

撤销之前，先判断改动在哪一层

所有示例只在一次性练习仓库中操作。

加错到暂存区

```
git restore --staged notes.txt
```

保留文件编辑，只撤销暂存。

不需要未暂存修改

```
git restore notes.txt
```

会丢弃工作区修改；确认备份再用。

撤销一个普通提交

```
git revert <commit>
```

新增反向提交，旧历史仍保留。

丢弃修改与改写历史前，先确认作用范围并备份；不要把强制命令当作通用修复。

冲突不是代码丢了，是 Git 等你决定

两个分支有提交不一定冲突；同一位置的修改可能无法自动兼容。

冲突文件示意

```
<<<<<< HEAD
title=main
=====
title=feature
>>>>>> feature/title
```

1 读懂双方意图

手工确定最终内容，删除冲突标记。

2 完成合并提交

```
git add title.txt
git commit -m "Resolve title conflict"
```

解决后是 `add` → `commit`。想退出本次合并：确认条件后 `git merge --abort`。

连不上 GitHub：先分辨是哪一类问题

不要把认证、仓库权限和网络超时混成一个“Git 不好用”。

publickey

检查 SSH 公钥、当前 agent 与问候账号。

not found

核对 owner / repo；私有仓库无权限也可能显示不存在。

连接超时

按网络条件选择 HTTPS + GCM 或官方 SSH 443 方式。

指纹不匹配

停止并核对官方信息，不关闭主机校验。

先确认错误类型，再检查认证、权限与网络；不要关闭主机校验或公开凭据。

推不上去：先看目录、分支与历史

多数问题先靠“看清状态”，不是靠强制覆盖。

排查时优先观察

```
pwd
git status
git branch -vv
git remote -v
git fetch origin
git --no-pager log --oneline \
  --graph --all -12
```

refspec 不存在

先有提交，再确认分支名。

non-fast-forward

远程和本地有分叉；
先观察历史，再选择整合。

不确定是不是同一个项目，就保留原目录，从正确远程新克隆一份再处理。

常见概念辨析

区分本地记录、远程同步、身份认证与合并行为。

pull $\neq$ 强制覆盖

先获取远程提交，再整合；`--ff-only` 在分叉时停止。 [2]

主分支变化 $\neq$ 必然冲突

自动整合不了的修改才需要人判断。 [3]

commit $\neq$ 上传

`commit` 写本地历史；`push` 更新远程分支。 [1][8]

邮箱与认证不是一回事

可用账号关联邮箱或完整 `noreply`；SSH / HTTPS 负责认证。 [12][13]

密钥身份 $\neq$ 提交作者

SSH 用于认证；`user.name` / `user.email` 写入提交记录。 [9]–[13]

main 是本例的主分支名；其他项目以实际分支为准。HTTPS 认证不使用网页登录密码。

参考资料

命令解释、认证流程与安全说明可进一步查阅下列资料。

课程参考

[A] Software Development Processes
Implementation & Deployment

[B] github 小白使用指南

参考章节与完整索引见《使用指南》

Git / GitHub

官方文档

Git：命令、分支与版本控制
GitHub：认证、托管与 Pull Request
账号安全与敏感数据保护

各页方括号编号对应《使用指南》末尾的参考资料索引。